WaterFlow DSL – Parser Documentation

1. Introduction

The WaterFlow DSL (.wflo) is a domain-specific language designed to model river systems and simulate water flows. It allows users to declare rivers and dams, connect them into systems, combine flows, and identify the output of the system. This assignment implements a parser in Java for this DSL, based on the structure from Crafting Interpreters (Ch. 3–6).

2. Grammar (Nystrom's Notation)

```
program       → declaration* EOF ;
declaration   → riverDecl | damDecl | letDecl | drainStmt | outputStmt ;
riverDecl     → "river" IDENT ( "=" NUMBER )? ";" ;
damDecl       → "dam" IDENT ";" ;
letDecl       → "let" IDENT "=" expression ";" ;
drainStmt     → IDENT "->" IDENT ";" ;
outputStmt    → "output" IDENT ";" ;
expression    → addition ;
addition      → primary ( "+" primary )* ;
primary       → IDENT | NUMBER | "(" expression ")" | rainLit ;
rainLit       → "rain" "(" NUMBER "mm" ")" ;
comment       → "//" <any chars until end of line> | "/*" <any chars until "*/">
```

3. How to Compile and Run

From the project root (with src/com/waterflo structure):

- Linux/macOS:

  ```
  javac -d out $(find src -name "*.java")
  ```

- Windows:

  ```
  javac -d out src\com\waterflo\*.java
  ```

- Run:

  ```
  java -cp out com.waterflo.Main examples/example1.wflo

  java -cp out com.waterflo.Main examples/example2.wflo

  java -cp out com.waterflo.Main examples/example3.wflo
  ```

4. Table A

Question	Answer
What literal represents a river that gets L/s of flow on the first day after 1 mm of rainfall?	rain(1mm)
What symbol is used to show two rivers combine?	+
Is the above symbol "unary", "binary", or "literal"?	Binary
What folder is the "working folder" to compile your parser?	The project root (waterflo/) containing src/com/waterflo/
What command(s) will compile your parser?	See compile commands above
How long does it take all the water to work through a river system after 1 day of rain?	Rainfall is assumed to propagate through the river system within approximately 10 days, with flows added to rivers immediately on the day they occur.
Does your language include statements or is it an expression language?	Statement language (with expressions inside let)
Which chapter of the book have you used as the starting point for your solution?	Crafting Interpreters up to Chapter 6 – Parsing Expressions

5. Example Programs

- Example 1 (simple):

```
river googong;
output googong;
```

- Example 2 (medium):

```
river a;
river b;
```

```
let c = a + b;
output c;
```

- Example 3 (complex – Canberra-style):

```
river googong = 120;
river jerrabombarra = 85;
river upper_molongolo;

dam dam1;
dam dam2;

river queanbeyan;
river lower_molongolo;

googong -> dam1;
dam1 -> queanbeyan;

let central = queanbeyan + upper_molongolo + jerrabombarra;
central -> dam2;
dam2 -> lower_molongolo;

let rain_today = rain(25mm);
output lower_molongolo;
```

6. How Parsing Works

When a .wflo file is scanned and parsed, the program is converted into an AST (Abstract Syntax Tree). For example, parsing example2.wflo:

```
river a;
river b;
let c = a + b;
output c;
```

Produces the AST:

```
RIVER name=a length=none
RIVER name=b length=none
LET name=c value=(+ a b)
OUTPUT name=c
```

7. Design Notes

• Optional river length: river googong = 120; allows initializing rivers with metadata (extensible for future flow simulations).

• Rainfall literals: rain(25mm) provides a way to express rainfall input.

• Combination operator: + used to model the confluence of rivers.

• Drain statements: a -> b; models topology connections.

• Comments: support both // and /* … */ to improve readability.

• File extension: .wflo chosen for clarity ("water flow").

8. AST Output (Screenshots)

```
PS C:\Users\patel\eclipse-workspace\Assignmen1_comp3000> javac -d out src\com\waterflo\*.java
PS C:\Users\patel\eclipse-workspace\Assignmen1_comp3000>
```

```
PS C:\Users\patel\eclipse-workspace\Assignmen1_comp3000> java -cp out com.waterflo.Main examples/example1.wflo
Stmt 1:
  River
    name: googong
    length: none

Stmt 2:
  Output
    name: googong
```

```
PS C:\Users\patel\eclipse-workspace\Assignmen1_comp3000\src> java -cp out com.waterflo.Main examples/example2.wflo
Stmt 1:
  River
    name: a
    length: none

Stmt 2:
  River
    name: b
    length: none

Stmt 3:
  Let
    name: c
    value: (+ a b)

Stmt 4:
  Output
    name: c
```

```
PS C:\Users\patel\eclipse-workspace\Assignmen1_comp3000> java -cp out com.waterflo.Main examples/example3.wflo
Stmt 1:
  River
    name: googong
    length: none

Stmt 2:
  River
    name: jerrabombarra
    length: none

Stmt 3:
  River
    name: upper_molongolo
    length: none

Stmt 4:
  Dam
    name: dam1

Stmt 5:
  Dam
    name: dam2

Stmt 6:
  River
    name: queanbeyan
    length: none

Stmt 7:
  River
    name: lower_molongolo
    length: none

Stmt 8:
  Drain
    from: googong
    to: dam1
```

```
Stmt 9:
  Drain
    from: dam1
    to: queanbeyan

Stmt 10:
  Let
    name: central
    value: (+ (+ queanbeyan upper_molongolo) jerrabombarra)

Stmt 11:
  Drain
    from: central
    to: dam2

Stmt 12:
  Drain
    from: dam2
    to: lower_molongolo

Stmt 13:
  Let
    name: rain_today
    value: (rain 25mm)

Stmt 14:
  Output
    name: lower_molongolo
```